SPRING DATA JPA — FUNDAMENTALS

1 Understand the Three Technologies

JPA (Jakarta Persistence)

- A specification (set of rules) that defines standard ORM and persistence rules in Java.
- It defines interfaces, annotations and behavior for mapping Java objects to database tables.

Hibernate

- A JPA implementation.
- It implements the JPA specification and provides the actual ORM engine.
- It generates and executes SQL based on the JPA rules.

Spring Data JPA

- A Spring abstraction that provides repository interfaces.
- It reduces repetitive database-access code and makes it easy to work with JPA and Hibernate.

JDBC (Java Database Connectivity)

- A lower-level Java API used to communicate with relational databases.
- It provides a set of APIs to execute SQL queries using a database driver.

2 The Complete Architecture

Application : Controller / Service
- Handles business logic
- Calls repository methods

→ e.g. REST Controller Service class

↓

Spring Data JPA : Repository Abstraction
- Provides repository interfaces (e.g. JpaRepository)
- Reduces repetitive database-access code

↓

JPA : Standard Persistence Interfaces
- Defines standard ORM and persistence rules
- Provides interfaces and annotations (e.g. Entity, Id, etc.)

↓

Hibernate : ORM Implementation
- Implements the JPA specification
- Maps entities to database tables
- Generates and executes SQL

↓

JDBC → MySQL / PostgreSQL Database
- Communicates with the database using SQL and database driver
- Database executes the SQL queries



MySQL / PostgreSQL

Each layer has a different responsibility.

3 How Saving a User Works

1. Service calls userRepository.save(user).
2. Spring Data JPA delegates to the persistence infrastructure.
3. Hibernate maps the entity fields to database columns and prepares SQL.
4. JDBC communicates with the database.
5. The database executes the INSERT; the generated ID is returned when supported.

Example:

```
userRepository.save(user);
```

4 Comparison Table

Technology	Role	Writes SQL?
JPA	Specification (Jakarta Persistence)	No
Hibernate	Implementation / ORM engine	Generates SQL
Spring Data JPA	Repository abstraction	Delegates to persistence provider
JDBC	Database communication API	Executes SQL through a driver

5 Why Use It ?

- Less boilerplate code.
- Repository methods and derived queries.
- Built-in CRUD, sorting, and pagination.
- Integration with Spring transactions.
- Database portability for standard JPA queries.

Interview Key Point :

> "JPA defines the contract. Hibernate implements it. Spring Data JPA simplifies how we use it."

SPRING BOOT — PROJECT SETUP

1 Essential Dependencies

Maven (pom.xml)

pom.xml

```xml
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Choose the driver for your database -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
```

1. **JPA starter:** Spring Data JPA, JPA APIs, Hibernate defaults, and transaction infrastructure.
2. **Database driver:** enables communication with the selected database.
3. Choose MySQL OR PostgreSQL driver, not both unnecessarily.

Gradle (build.gradle)

```
implementation 'org.springframework.boot:
                spring-boot-starter-data-jpa'
```

2 Database Configuration

application.properties

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=yourpassword

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

- **spring.datasource.url**
  - Identifies the database location and name.
- **spring.datasource.username / password**
  - Credentials authenticate the connection.
- **spring.jpa.hibernate.ddl-auto**
  - Defines how Hibernate should manage the database schema.
- **spring.jpa.show-sql=true**
  - Helps inspect the generated SQL in the console during development.
- **spring.jpa.properties.hibernate.format_sql=true**
  - Improves SQL readability by formatting the output.

application.properties and application.yml configure the same settings using different syntax.

3 Application Startup Flow

1. **Read configuration.**
   - Loads properties from application.properties / yml.
2. **Create the database connection pool.**
   - Uses datasource settings and creates a connection pool (e.g. HikariCP).
3. **Initialize Hibernate and inspect entity mappings.**
   - Scans entities and builds metadata from JPA annotations.
4. **Apply the configured schema strategy.**
   - Uses ddl-auto (create / update / validate / etc.) to manage schema.
5. **Discover repositories and create repository implementations.**
   - Spring Data JPA creates proxies for repository interfaces.
6. **Application becomes ready.**
   - Application context is fully initialized and ready to handle requests.

4 DDL-AUTO QUICK REFERENCE

Value	Description
create	Recreate schema at startup. Existing data can be lost.
create-drop	Create schema at startup and drop it at application shutdown.
update	Adjust schema based on entity changes (useful for development). Not a production migration strategy.
validate	Verify that the schema matches the entities. Application fails if there are mismatches.
none	Do not manage the schema automatically. You must create and manage the schema manually.

Production Note:
Manage schema changes with Flyway or Liquibase. Never rely on create/update as your production migration process.

Example (application.yml)

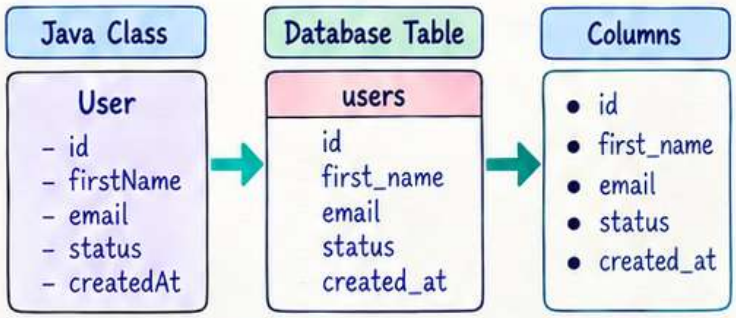
application.yml

```yaml
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb
    username: root
    password: yourpassword
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
```

ENTITY MAPPING — FROM JAVA TO DATABASE

1 The Core Idea

- **Entity class** → database table.
- **Entity object** → one row.
- **Persisted field** → database column, unless explicitly excluded or otherwise mapped.



2 Compact Entity Example

User.java

```java
@Entity
@Table(name = "users")
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name",
            nullable = false, length = 50)
    private String firstName;

    @Column(nullable = false, unique = true,
            length = 100)
    private String email;

    @Enumerated(EnumType.STRING)
    private UserStatus status;

    @Transient
    private String displayLabel;

    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;
}
```

UserStatus.java

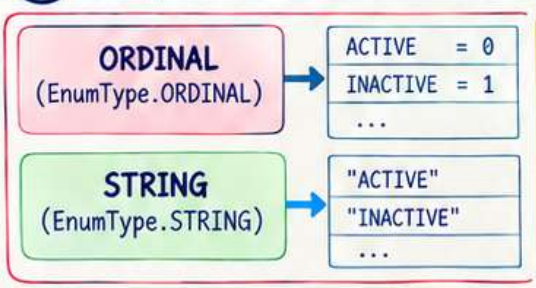
```java
enum UserStatus {
    ACTIVE, INACTIVE, BANNED
}
```

3 Annotation Reference

Annotation	Purpose
@Entity	Marks a persistent entity.
@Table	Customizes table mapping.
@Id	Identifies the primary key.
@GeneratedValue	Configures ID generation.
@Column	Configures column mapping.
@Enumerated (EnumType.STRING)	Stores enum names as text.
@Transient	Excludes a field from persistence mapping.
@Index	Supports faster lookups for suitable queries.
@UniqueConstraint	Enforces uniqueness in the database schema.
@NoArgsConstructor	Provides a no-argument constructor.

A JPA entity needs a **primary key** and a suitable **no-argument constructor.**
Lombok can generate the constructor.

4 Important Pitfall — Enum Storage



- Reordering enum constants can corrupt the meaning of ordinal-based stored values.
- Prefer STRING for maintainable business enums.

5 Interview Memory Line

> "Annotations define how the Java entity corresponds to the relational schema."

@GeneratedValue — ID GENERATION

1 Why It Exists

- A primary key uniquely identifies a database row; ID generation lets the database or persistence provider assign that value.

2 Four Strategies

1 IDENTITY

- Database identity/auto-increment.
- Common choice for MySQL.

```
@Id
@GeneratedValue(strategy =
  GenerationType.IDENTITY)
private Long id;
```



AUTO_INCREMENT
id: 1, 2, 3, ...

Trade-off: Hibernate generally cannot batch inserts using generated identity IDs in the same way as preallocated IDs.

2 SEQUENCE

- Database sequence.
- Common for PostgreSQL and Oracle.

```
@Id
@GeneratedValue(strategy =
   GenerationType.SEQUENCE,
   generator = "user_seq")
@SequenceGenerator(
  name = "user_seq",
  sequenceName = "user_sequence",
  allocationSize = 50
)
private Long id;
```



SEQUENCE
id: 1, 2, 3, ...
(from DB sequence)

Trade-off: allocationSize can reduce sequence-fetch round trips when configured correctly.

3 UUID

- Universally unique identifier.
- Useful for distributed systems where services generate IDs independently.

```
@Id
@GeneratedValue(strategy =
  GenerationType.UUID)
private UUID id;
```

550e8400-e29b-41d4-a716-446655440000

Trade-off: Generally larger identifiers and potentially larger indexes.

4 TABLE

- Uses a separate table to manage ID allocation.
- More overhead; uncommon for new high-performance applications.

```
@Id
@GeneratedValue(strategy =
  GenerationType.TABLE)
private Long id;
```



ID_TABLE
next_val: 1, 2, 3, ...

Uses a dedicated table (e.g. hibernate_sequences) to store and manage the next identifier value.

3 Strategy Comparison

Strategy	Typical Fit	Main Trade-off
IDENTITY	Simple applications / MySQL	Insert batching limitation.
SEQUENCE	PostgreSQL / Oracle and bulk inserts	Requires sequence support and configuration.
UUID	Distributed systems	Larger identifiers and potentially larger indexes.
TABLE	Compatibility scenarios	Extra allocation overhead.

4 Quick Decision Guide

MySQL auto-increment preference → IDENTITY

Sequence support + bulk-insert performance → SEQUENCE

Independent ID generation across services → UUID

Key Point:

Choose based on database support, identifier requirements, and measured workload — not on a universal rule.

JPA RELATIONSHIPS — WHO OWNS THE FOREIGN KEY?

1 Core Vocabulary

- **Owner side** : defines the database relationship, usually through the foreign key or join-table mapping.
- **Inverse side** : uses **mappedBy** to refer to the relationship field owned by the other entity.
- **@JoinColumn** : names the foreign-key column.
- **mappedBy** : references the Java field on the owning side.

2 One-to-One : User and UserProfile

User (Owner Side) — User.java

```
@Entity
class User {
    @Id
    private Long id;

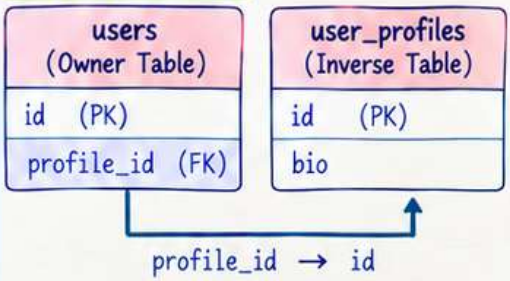
    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "profile_id")
    private UserProfile profile;
}
```

UserProfile (Inverse Side) — UserProfile.java

```
@Entity
class UserProfile {
    @Id
    private Long id;

    @OneToOne(mappedBy = "profile")
    private User user;
}
```

Database Tables



- **User is the owner** in this example. mappedBy does not create a second foreign key.

> LAZY for one-to-one relationships depends on mapping and provider support; verify actual loading behavior.

3 One-to-Many / Many-to-One : Department and Employee

Department (Inverse Side) — Department.java

```
@Entity
class Department {
    @Id
    private Long id;

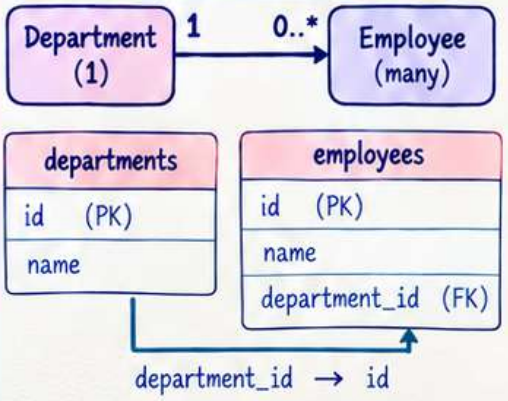
    @OneToMany(mappedBy = "department",
               fetch = FetchType.LAZY)
    private List<Employee> employees =
                new ArrayList<>();
}
```

Employee (Owner Side) — Employee.java

```
@Entity
class Employee {
    @Id
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id",
                nullable = false)
    private Department department;
}
```

Relationship & Database Tables



- The foreign key lives in the many-side table, so **Employee** is the owner.

4 Keep Both Sides in Sync (Bidirectional Relationship)

Department.java

```
public void addEmployee(Employee employee) {
    employees.add(employee);
    employee.setDepartment(this);
}
```

- Update both Java references in a bidirectional relationship to keep the in-memory model consistent.
- This ensures that both sides reflect the relationship correctly before saving.
- Helps avoid unexpected behavior when working with the persistence context.

MANY-TO-MANY + ENTITY LIFECYCLE RULES

1 Many-to-Many Example

Scenario: A student can attend many courses; a course can have many students.

Student (Owner Side)

Student.java

```
@Entity
class Student {
    @Id
    private Long id;

    @ManyToMany(fetch = FetchType.LAZY,
                cascade = {CascadeType.PERSIST,
                           CascadeType.MERGE})
    @JoinTable(
        name = "student_courses",
        joinColumns = @JoinColumn(name = "student_id"),
        inverseJoinColumns = @JoinColumn(name = "course_id")
    )
    private Set<Course> courses = new HashSet<>();
}
```

Course (Inverse Side)

Course.java

```
@Entity
class Course {
    @Id
    private Long id;

    @ManyToMany(mappedBy = "courses",
                fetch = FetchType.LAZY)
    private Set<Student> students = new HashSet<>();
}
```

- Student owns the join-table mapping.
- Course is the inverse side.
- Set prevents duplicate membership at the Java collection level.

2 Join-Table Diagram and Example Data

students

id (PK)	name
1	Alice
2	Bob

1 → Many

student_courses

student_id (FK)	course_id (FK)
1	101
1	102
2	101

Many ← 1

courses

id (PK)	title
101	Math
102	Science

Example Rows (Meaning)

- Alice (id=1) enrolled in Math (101) and Science (102).
- Bob (id=2) enrolled in Math (101).
- Each row in student_courses represents one enrollment.

3 Cascade Types

Cascade Type	Purpose
PERSIST	Persist children with parent.
MERGE	Propagate merge operations.
REMOVE	Remove children with parent.
REFRESH	Refresh related state.
DETACH	Detach related state.
ALL	All listed cascade operations.

4 Orphan Removal (orphanRemoval = true)

Parent collection contains child
`parent.getChildren()` [child1, child2]

→ Child removed from collection
`parent.getChildren()` [child1]

→ orphanRemoval = true
Child deletion is scheduled.

- **orphanRemoval** applies when a child is removed from the parent's relationship collection. The removed child is treated as an orphan and scheduled for deletion.
- **CascadeType.REMOVE** applies when the parent is removed. It removes the child entities along with the parent.

5 Important Rules and Best Practices

⚠ Never casually cascade REMOVE or ALL across shared many-to-many entities. Deleting one Student must not delete a Course still used by other students.

- Use PERSIST and MERGE for many-to-many in most cases.
- Do not rely on cascade to replace database constraints. Foreign keys and constraints still apply.
- Choose the owning side intentionally (the side that defines @JoinTable).

When to Model the Join Table as an Entity?

- If the join table needs additional attributes such as enrollmentDate, grade, quantity, etc.
- Model it as a separate entity (e.g. StudentCourse) with two @ManyToOne relationships:
  - @ManyToOne Student
  - @ManyToOne Course
- This gives you full control over the join table data and allows adding business logic.

FETCH TYPES — WHEN DOES RELATED DATA LOAD?

1 Understand the Difference

EAGER

- Related data must be available as part of fetching the entity.
- This does not guarantee one SQL JOIN; the provider may use additional queries.

LAZY

- Related data is fetched when it is accessed, while the persistence context can still load it.
- Helps avoid loading unnecessary data.
- Uses a proxy (for single-valued) or a lazy collection (for collections).

2 Defaults to Memorize

Relationship	Default Fetch Type
@ManyToOne	EAGER
@OneToOne	EAGER
@OneToMany	LAZY
@ManyToMany	LAZY

Prefer LAZY mappings where appropriate, then fetch what the use case actually needs.

3 LAZY Loading Sequence (Example: Department → Employees)

1 findById(1L)

```
Department dept =
repository.findById(1L)
    .orElseThrow();
```

No SQL for employees yet.

2 SELECT department row

```
SELECT * FROM departments
WHERE id = 1;
```

departments	
id	name
1	IT

Department entity is loaded.

3 employees as proxy/collection

dept.getEmployees() has not been called yet, so the employees collection is an unloaded proxy (lazy collection).

Department dept → employees (proxy / lazy collection)

4 Access within open context

```
dept.getEmployees()
```

```
SELECT * FROM employees
WHERE department_id = 1;
```

employees		
id	name	department_id
1	John	1
2	Riya	1

Employees are loaded when accessed.

4 The Common Failure — LazyInitializationException

Transaction / persistence context closes → Entity becomes detached → Code accesses an unloaded lazy relationship → LazyInitializationException

```
Department department = repository.findById(1L)
                        .orElseThrow();

department.getEmployees();  // Risk if the required
                            // persistence context is closed
```

Why this happens?

- Lazy loading needs an active persistence context to fetch data from the database. After the context is closed, the proxy cannot initialize, leading to LazyInitializationException.
- Common outside a transactional service method (e.g., in controllers or after the session is closed).

5 Better Practice — Load Intentionally

```
@Query("""
      SELECT e FROM Employee e
      JOIN FETCH e.department
      """)
List<Employee> findAllWithDepartment();
```

- Use JOIN FETCH or @EntityGraph when a specific use case needs related data.
- This loads the required association in a single query (or optimized queries).
- Avoid changing everything to EAGER just to hide LazyInitializationException.
- Load only what is needed for the use case.

REPOSITORY LAYER — YOUR DATABASE ACCESS API

1 What Is a Repository?

- An interface that exposes data-access operations. Spring Data creates the runtime implementation; developers normally declare the interface rather than implement every CRUD method.



2 Repository Capabilities

Different repository interfaces provide different sets of capabilities (Modern Spring Data).

Repository<T, ID> (marker / root abstraction) — Base interface for all repositories.

CrudRepository<T, ID>
- Basic CRUD operations (save, find, delete, etc).

ListCrudRepository<T, ID>
- Same as CrudRepository but returns List where applicable.

PagingAndSortingRepository<T, ID>
- Pagination and sorting operations.

ListPagingAndSortingRepository<T, ID>
- Pagination and sorting operations returning List where applicable.

JpaRepository<T, ID>
- Combines list-based CRUD and paging/sorting capabilities.
- Adds JPA-specific operations.

Note: Older Spring Data versions had different inheritance relationships.

3 Repository Example

UserRepository.java

```
public interface UserRepository
        extends JpaRepository<User, Long> {
    // No need to implement methods.
    // Spring generates the implementation at runtime.
}
```

- User = entity type (the JPA entity managed by this repository).
- Long = ID type (type of the entity's primary key).
- Spring generates the implementation at runtime.

4 Common Built-In Methods

Create / Update
- save(user)
  - Saves a new entity or updates an existing entity.
- saveAll(users)
  - Saves multiple entities.

Read
- findById(id)
  - Returns Optional<T>.
- findAll()
  - Returns all entities.
- count()
  - Returns total count.
- existsById(id)
  - Checks if an entity exists.

Delete
- deleteById(id)
  - Deletes by ID.
- delete(user)
  - Deletes the given entity.
- deleteAllInBatch()
  - Deletes all entities in a single query (without loading them).

5 Interview Pitfalls & Key Points

1. **findById()** returns Optional<User>, not User.
   - Handle missing data explicitly.
2. **save()** is commonly used for both new and existing entities, but exact new-entity detection depends on Spring Data's rules and entity mapping.
3. **Bulk delete** methods (e.g., deleteAllInBatch()) can bypass entity loading, cascades, and lifecycle callbacks.
4. **getReferenceById()** may return a proxy rather than immediately fetching all entity data.

Correct Handling of findById()

```
User user = userRepository.findById(1L)
        .orElseThrow(() -> new RuntimeException
                ("User not found"));
```

Key Takeaway
- Use the appropriate repository interface based on your needs.
- Let Spring Data generate the implementation.
- Understand the behavior of built-in methods, especially for delete and proxy-based methods.

DERIVED QUERIES — LET THE METHOD NAME DESCRIBE THE FILTER

1 Parse This Method

findBy FirstName And Status OrderBy CreatedAtDesc

Query prefix
- findBy / readBy / queryBy / getBy
- Indicates a select query

First condition
- Property: firstName
- Equivalent to first_name = ?

Conjunction
- Combines conditions using AND

Second condition
- Property: status
- Equivalent to status = ?

Sorting
- Order by property: createdAt
- Descending order (DESC)

Conceptual SQL:

```
SELECT ... FROM users
 WHERE first_name = ?
       AND status = ?
 ORDER BY created_at DESC;
```

Spring parses and validates derived-query property paths during repository initialization. If a property is not found, the application will fail to start.

2 Useful Derived Query Method Patterns

Some commonly used examples in real applications.

Method Signature	Purpose
`Optional<User> findByEmail(String email);`	Find a user by email.
`List<User> findByStatus(UserStatus status);`	Find users by status.
`List<User> findByAgeBetween(int min, int max);`	Find users within an age range.
`List<User> findByFirstNameContaining(String keyword);`	Find users whose first name contains a keyword.
`List<User> findByBioIsNull();`	Find users with bio as null.
`List<User> findByStatusIn(List<UserStatus> statuses);`	Find users with status in a given list.
`List<User> findByStatusOrderByCreatedAtDesc(UserStatus status);`	Find users by status, ordered by creation date (desc).
`List<User> findTop5ByStatusOrderByCreatedAtDesc(UserStatus status);`	Find top 5 users by status, ordered by creation date.
`long countByStatus(UserStatus status);`	Count users by status.
`boolean existsByEmail(String email);`	Check if a user exists by email.

3 Keyword Quick Reference

Keyword(s)	Meaning / Generated Condition
And / Or	Combine multiple predicates (AND / OR).
LessThan / GreaterThan	Comparison (< , >).
Between	Range comparison (BETWEEN).
Containing	LIKE %value% (for strings).
StartingWith	LIKE value% (for strings).
EndingWith	LIKE %value (for strings).
IsNull / IsNotNull	Check for null values.
In / NotIn	Membership check (IN / NOT IN).
IgnoreCase	Case-insensitive comparison.
OrderBy	Ordering (ASC or DESC).
Top / First	Limit the number of results.

4 Common Mistakes & Best Practices

1. **Property names must match the entity's Java properties.**
   Use the exact field name (e.g., firstName, not first_name).
2. **Misspelled names can cause startup errors.**
   Spring will fail to create the repository bean if the property cannot be resolved.
3. **Nested property paths can be ambiguous.**
   Use an underscore (_) to resolve, e.g., `findByDepartment_Name(String name)`.
4. **Switch to @Query for complex conditions.**
   When method names become long or hard to understand, a custom @Query is more readable and maintainable.

Key Takeaway

Derived queries are great for simple to moderate queries. For complex queries, prefer @Query with JPQL or native SQL.

PAGINATION & SORTING — EFFICIENT DATA ACCESS

1 Key Concepts

- **Pageable** : Contains page number, size and optional sort information.
- **PageRequest** : A Pageable implementation used to create page requests.
- **Sort** : Specifies the field(s) to sort by and the direction (ASC or DESC).
- **Page<T>** : Contains records plus pagination metadata (e.g., totalElements, totalPages). Usually requires a count query to get total count.
- **Slice<T>** : Contains records plus hasNext information, without requiring a total-count result.

2 Repository Example

UserRepository.java

```
public interface UserRepository
     extends JpaRepository<User, Long> {

    Page<User> findByStatus(
         UserStatus status, Pageable pageable);
}
```

3 Service Example

```
Pageable pageable = PageRequest.of(
    0, 10,
    Sort.by("createdAt").descending()
);

Page<User> users =
    userRepository.findByStatus(
        UserStatus.ACTIVE, pageable);
```

- page number = 0 (first page, zero-indexed)
- page size = 10 (records per page)
- sort = createdAt DESC

4 REST URL Example (Spring Boot Endpoint)

GET /api/users?page=0&size=10&sortBy=createdAt&direction=desc

Client query parameters	Controller (@RequestParam)	PageRequest / Pageable	Repository	Database Query	Page<User> Response
page = 0 size = 10 sortBy = createdAt direction = desc	• Read query parameters • Create PageRequest / Pageable • Call service/repository	• Page number: 0 • Page size: 10 • Sort: createdAt DESC	findByStatus(status, pageable)	• Applies LIMIT & OFFSET for page size and number • Orders by createdAt DESC	• Records for current page • Pagination metadata

ℹ Page numbers are zero-indexed: page = 0 is the first page.

5 Page vs Slice

Page<T>	Slice<T>
• Contains content (records). • Provides totalElements, totalPages, current page number, page size, etc. • A count query is commonly needed to get the total count. • Useful for numbered-page interfaces (e.g., page 1, page 2, ...).	• Contains content (records). • Provides hasNext (and usually number, size). • Does not require a total-count result. It fetches one extra record to determine hasNext. • Useful for load-more or infinite-scroll use cases.

💡 Use **Page** for numbered-page interfaces; use **Slice** for load-more or infinite-scroll scenarios. Choose based on your use case and the cost of the count query.

6 Sample Response (Page<User>)

JSON Response

```
{
  "content": [
    { "id": 1, "firstName": "John", "status": "ACTIVE" }
  ],
  "totalElements": 150,
  "totalPages": 15,
  "number": 0,
  "size": 10,
  "first": true,
  "last": false
}
```

Note:

- content → records of current page
- totalElements → total number of matching records
- totalPages → total number of pages
- number → current page number (zero-indexed)
- size → number of records per page

QUERY MECHANISMS — CHOOSE THE RIGHT TOOL

Spring Data JPA Notes

- Helps a developer choose among derived queries, JPQL, native SQL and named queries, understand parameter binding, and use data-modifying queries correctly.

1 JPQL vs Native SQL

JPQL (Java Persistence Query Language)	Native SQL
• References entity classes and Java properties.	• References database table and column names.
• Example: `SELECT u FROM User u WHERE u.email = :email`	• Example: `SELECT * FROM users WHERE email = :email`
• Portable across databases (when using standard JPQL).	• Supports database-specific features (e.g. window functions, CTEs, vendor-specific syntax).
• Best for common filtering, joins, aggregates and entity queries.	• Less portable; SQL errors may only appear at runtime.

JPQL Repository Example

```
@Query("SELECT u FROM User u WHERE u.email = :email")
Optional<User> findByEmail(
    @Param("email") String email);
```

Key Points (JPQL)

- Uses entity name (User) and Java properties (email).
- Returns managed entity objects.
- Portable across databases.

2 Named Queries

Entity with Named Query

```
@Entity
@NamedQuery(
   name = "User.findActive",
   query = "SELECT u FROM User u WHERE u.status = 'ACTIVE"
)
public class User {
   // fields, getters, setters
}
```

- A named query is registered on the entity or in mapping metadata and referenced by its name, conventionally associated with the entity and method.

Query Resolution (Important)

1. Method-level @Query takes precedence over a named query.
2. Otherwise, Spring may resolve a named query.
3. If no declared query is found, it can derive the query from the method name under the standard lookup strategy.

3 Modifying Queries (Update / Delete)

```
@Modifying(clearAutomatically = true)
@Tramacational
@Query("""
      UPDATE User u
      SET u.status = :newStatus
      WHERE u.status = :oldStatus
  """)
int updateStatus(
    @Param("oldStatus") UserStatus oldStatus,
    @Param("newStatus") UserStatus newStatus);
```

→ **@Modifying** → Identifies a modifying query (INSERT, UPDATE, DELETE).

→ **@Transactional** → Provides transaction management for the modifying operation.

→ **clearAutomatically = true** → Clears potentially stale managed state from the persistence context after execution.

4 Query Choice Guide

- Simple filters → Derived queries (method name based).
- Complex entity-oriented queries → JPQL.
- Database-specific operations → Native SQL.
- Reusable entity-associated query definitions → Named queries, where appropriate.

5 Important Rules

1. Prefer named parameters for readability.
2. Never concatenate untrusted values into SQL strings.
3. Native paginated queries may need an explicit countQuery.
4. Test native SQL against the target database.

PROJECTIONS — LIGHTWEIGHT READ MODELS

Spring Data JPA Notes

- Projections allow you to fetch only the data you need instead of loading the complete entity. They are useful for read use cases, API response shaping, and avoiding unnecessary data transfer.

1 The Problem

Full User entity in database

User (Entity / Table)	
Column	**Value (Example)**
id	1
firstName	Sanket
lastName	Harvande
email	sanket@example.com
passwordHash	$2a$10$....
bio	Software Engineer...
status	ACTIVE
createdAt	2024-10-01 10:00:00

API needs only a few fields

`GET /users?status=ACTIVE`

Response needed (only 3 fields)

```
{
  "id": 1,
  "firstName": "Sanket",
  "email": "sanket@example.com"
}
```

Why retrieve every field when the endpoint needs only three?

Benefits

- Fetch only required columns from the database.
- Avoid transferring unnecessary data like passwordHash, bio, etc.
- Cleaner and smaller API responses.
- Reduced accidental exposure of sensitive fields.

2 Interface Projection

Spring Data can create a projection backed by the selected properties. Interface getter names must correspond to the intended entity properties.

Projection Interface

```
public interface UserSummary {
    Long getId();
    String getFirstName();
    String getEmail();
}
```

Repository Method

```
List<UserSummary> findByStatus(
    UserStatus status);
```

3 DTO-Based Projection

The DTO constructor or record constructor must match the selected values and their order.

DTO (Java Record)

```
public record UserSummaryDto(
    Long id, String firstName, String email) {
}
```

Repository Method with JPQL

```
@Query("""
    SELECT new com.example.dto.UserSummaryDto(
        u.id, u.firstName, u.email)
    FROM User u
    WHERE u.status = :status
""")
List<UserSummaryDto> findSummaries(
    @Param("status") UserStatus status);
```

4 Dynamic Projection

The caller chooses the requested projection type at runtime.

Repository Method

```
<T> List<T> findByStatus(
    UserStatus status, Class<T> type);
```

Example Usage

```
List<UserSummary> summaries =
    userRepository.findByStatus(UserStatus.ACTIVE,
      UserSummary.class);
List<UserSummaryDto> dtos =
    userRepository.findByStatus(UserStatus.ACTIVE,
      UserSummaryDto.class);
```

5 Comparison

Type	What it is	Key Points	Use Case
Interface Projection	Simple read-only view using an interface.	• Only selected properties • Getter names match entity properties	Lightweight API responses, simple use cases.
DTO Projection	Explicit result object (class/record).	• Uses JPQL constructor expression • Constructor must match selected values and order	When you want a custom object shape with precise control.
Dynamic Projection	Caller-selected result shape.	• Method accepts Class<T> type • Choose projection type at runtime	Flexible APIs supporting multiple projection types.

6 Real-World Advantages

- Less data transfer between database and application.
- Lower memory overhead in many read scenarios.
- Cleaner API response shaping.
- Reduced accidental exposure of sensitive fields when those fields are excluded from the result.

7 Interview Note

Use entities when you need managed domain objects (for updates, associations, or business logic).
Use projections for focused read use cases where only specific fields are required.

THE N+1 PROBLEM – ONE QUERY BECOMES MANY

- Explain why N+1 queries occur, how to recognize them, compare major solutions, and demonstrate why loading a collection of entities does not guarantee all related entities are loaded efficiently.

1 The Problem

1 Query → Load N employees

N Extra Queries → Load departments as accessed

Total → 1 + N queries

⚠ Lazy relationship access inside a loop can trigger repeated database round trips.

Before Optimization: N+1 Scenario

1 Query (Employees) + N Queries (Departments)

Employees (N rows)

id	name	dept_id
1	Alice	101
2	Bob	102
3	Charlie	101
...	...	...

```
SELECT * FROM department
WHERE id = ?
```

(repeated N times)

Total: 1 + N Queries

2 Problematic Code

```
List<Employee> employees =
    employeeRepository.findAll();

for (Employee employee : employees) {
    System.out.println(
        employee.getDepartment().getName());
}
```

If department data is fetched separately for each employee, the query count can grow toward 1 + N.

3 Solution A: JOIN FETCH

```
@Query("""
    SELECT e FROM Employee e
    JOIN FETCH e.department
    """)
List<Employee> findAllWithDepartment();
```

One Combined Result

Employees (from employees table) + Departments (from department table)

✓ Loads Employee and Department in a single query

4 Other Solutions

@EntityGraph (attributePaths = {"department"})

→ Declaratively specify relationships to load for a repository method.

@BatchSize(size = 30)

→ Group relevant relationship loads into batches; query count is reduced, not necessarily eliminated.

@Fetch(FetchMode.SUBSELECT)

→ Hibernate-specific option that may fetch collections using a subselect.

5 Choose the Solution

- ✓ **JOIN FETCH** → common targeted fix.
- ✓ **@EntityGraph** → convenient with existing repository methods.
- ✓ **@BatchSize** → helpful when batching is preferable to large fetch joins.
- ✓ **SUBSELECT** → use when its collection-loading behavior fits the access pattern.

6 Detection

spring.jpa.show-sql=true

```
Hibernate: select e1_0.id, e1_0.name from employee e1_0
Hibernate: select ... from department d1_0
employee
    ... d1_0.id, d1_0.n...
    where d1_0.id=?
Hibernate:
departmena
    select d1_0.id, d1_0.name from department d1_0
    department d1_0
    where d1_0.id=?
Hibernate:
empdrmeee
    select d1_0.id, d1_0.name from department d1_0
    where d1_0.id=?
...
```

💡 Inspect query logs or use query-count tools in tests.

★ **Key Takeaway:** The N+1 problem occurs when lazy loading triggers repeated queries. Use JOIN FETCH, @EntityGraph, @BatchSize or SUBSELECT to fetch related data efficiently.

ENTITY LIFECYCLE + AUTO-SAVING CHANGES

Spring Data JPA Notes

- JPA entities go through different states during their lifecycle. Hibernate tracks changes of managed entities and automatically synchronizes them with the database (dirty checking).

1 Four Entity States (Lifecycle)

TRANSIENT
- Ordinary new Java object.
- Not managed by the persistence context.
- Not associated with a database record.

`User user = new User();`

persist() (or save())

Entity becomes managed (insert will happen on flush/commit).

MANAGED
- Tracked by the persistence context (1st level cache).
- Changes can be detected (dirty checking).
- Synchronized with the database at flush/commit.

`entityManager.persist(user)` (or `repository.save(user)`)

Transaction ends or detach() / clear()

DETACHED
- Previously managed, but no longer tracked by that persistence context.
- Changes are NOT automatically synchronized.

`entityManager.detach(user)` (or session ends)

remove()

REMOVED
- Managed entity scheduled for deletion.
- Actual DELETE happens at flush/commit.

`entityManager.remove(user)` (or `repository.delete(user)`)

merge(detached) → returns managed copy/state in a new persistence context.

⭐ Spring Data **save()** chooses the appropriate persistence operation (**persist** or **merge**) based on the entity state and identifier. Do not assume every returned instance has identical managed-state behavior in every mapping scenario.

2 How Dirty Checking Works?

1. Hibernate loads a managed entity from the database.
2. It keeps a snapshot of the entity's original state in the persistence context.
3. Application code changes a field of the managed entity.
4. At flush time (e.g., before transaction commit), Hibernate detects the difference.
5. It generates the required UPDATE statements.
6. The SQL is synchronized with the database within the transaction.

Load Entity (Managed)

```
SELECT *
FROM user
WHERE id = ?
```

→

Modify Field (in Java code)

```
user.setEmail(
"new@example.com");
```

→

Flush / Commit (Dirty Checking)

```
UPDATE user
SET email = ?
WHERE id = ?
```

3 Code Example (No Explicit save)

```
@Transactional
public void updateEmail(Long id, String email) {
    User user = userRepository.findById(id)
            .orElseThrow();

    user.setEmail(email);  // only changing the field
    // no explicit save() needed
}
```

No explicit **save()** call is required for a managed entity changed within this transaction; dirty checking handles persistence synchronization.

4 What Fails?

DETACHED Object

```
User user = ...
// loaded earlier
// now detached
```

user.setEmail(...) ✗

Database is NOT automatically updated.

Database

No UPDATE happens automatically for detached objects.

5 Optimization - Update Only Changed Columns

```
@Entity
@DynamicUpdate
class User {
    // mapped fields
}
```

- Hibernate can generate UPDATE statements containing only the changed columns rather than every column.
- This can reduce the amount of data sent to the database.
- Evaluate the trade-offs for your actual workload (some databases and use cases may not see significant benefits).

6 Key Points

- Only managed entities are automatically tracked for changes.
- Detached entity changes are not saved without merge().
- Dirty checking works at flush/commit time.
- Use @DynamicUpdate when appropriate.
- Consider the entity lifecycle when designing service methods and transaction boundaries.

TRANSACTIONS — COMMIT OR ROLLBACK

Spring Data JPA Notes

- A transaction groups related database operations into one unit of work. Either the transaction commits successfully or its database changes roll back according to the configured rules.

1 The Core Idea (Example: Bank Transfer)

Transfer ₹1000 from Account A to Account B

Account A (balance: 5000) → Account B (balance: 2000)

1. Debit ₹1000 from Account A
2. Credit ₹1000 to Account B

Both operations succeed
✓ **COMMIT**
Changes are permanently saved in the database.
Account A (balance: 4000) | Account B (balance: 3000)

If any operation fails
✗ **ROLLBACK**
All database changes made in this transaction are undone.
Account A (balance: 5000) | Account B (balance: 2000)

⚠ An ordinary database transaction does not automatically undo a charge performed by an external payment API or another independent service.

2 Transaction Flow (High-Level)

External Caller (e.g. Controller) → Spring Transactional Proxy (@Transactional) → Begin Transaction (start DB transaction) → Service Method (Business Logic) + Repository Operations (perform DB operations)

- Normal Return → COMMIT (commit transaction)
- Matching Exception → ROLLBACK (rollback transaction)

3 Common Transactional Attributes

Attribute	Purpose
readOnly = true	Declares a read-oriented transaction; can provide provider-specific optimizations.
rollbackFor = Exception.class	Include checked exceptions in rollback rules.
timeout = 30	Set a transaction timeout in seconds where supported.
propagation = REQUIRED	Join existing transaction or start one (if none exists).

★ By default, Spring rolls back on unchecked exceptions (RuntimeException) and Errors. Checked exceptions generally do not trigger rollback without additional rules.

4 Propagation Behavior

Propagation	Behavior
REQUIRED (default)	Join existing transaction or start a new one if none exists.
REQUIRES_NEW	Suspend current transaction and start a separate new transaction.
SUPPORTS	Join existing transaction if available, otherwise proceed without a transaction.
MANDATORY	An existing transaction is required; throws an exception if none exists.
NEVER	Execute without a transaction; throws an exception if a transaction exists.
NOT_SUPPORTED	Suspend any existing transaction and run without a transaction.
NESTED	Run within a transaction using a savepoint (if supported by the underlying database).

5 Common Pitfalls

1. Put transactional business boundaries on service-layer methods.
2. Private method calls are not intercepted by a standard proxy.
3. Self-invocation (calling another method of the same class) can bypass proxy-based transaction handling.
4. Keep database transactions appropriately scoped (avoid very long transactions).
5. Use DTOs or explicit fetch plans instead of depending on detached lazy entities.

6 Code Example

```
@Transactional
public void transfer(Long fromId, Long toId, BigDecimal amount) {
    Account from = accountRepository.findById(fromId).orElseThrow();
    Account to = accountRepository.findById(toId).orElseThrow();

    from.debit(amount);   // custom method
    to.credit(amount);    // custom method
}
```

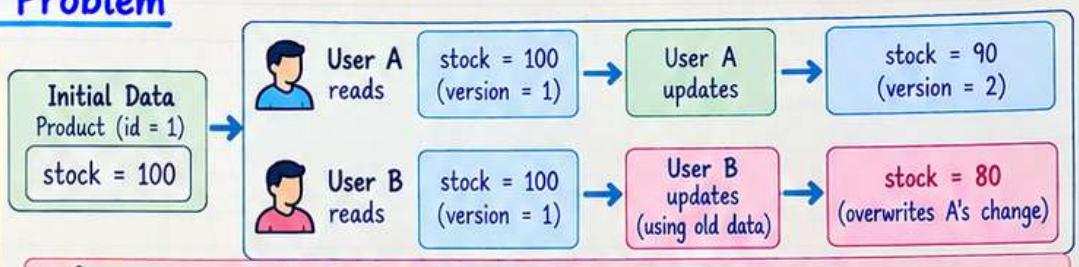
OPTIMISTIC VS PESSIMISTIC LOCKING

Spring Data JPA Notes

- Locking helps maintain data consistency when multiple transactions access and modify the same data concurrently. It prevents problems such as the lost-update problem.

1 The Lost-Update Problem

Two users read the same data and update it. Without proper locking, a later write can overwrite an earlier change.



⚠ Without protection, the second update can overwrite the first one, resulting in a lost update.

2 Optimistic Locking (Using @Version)

Entity with Version Field

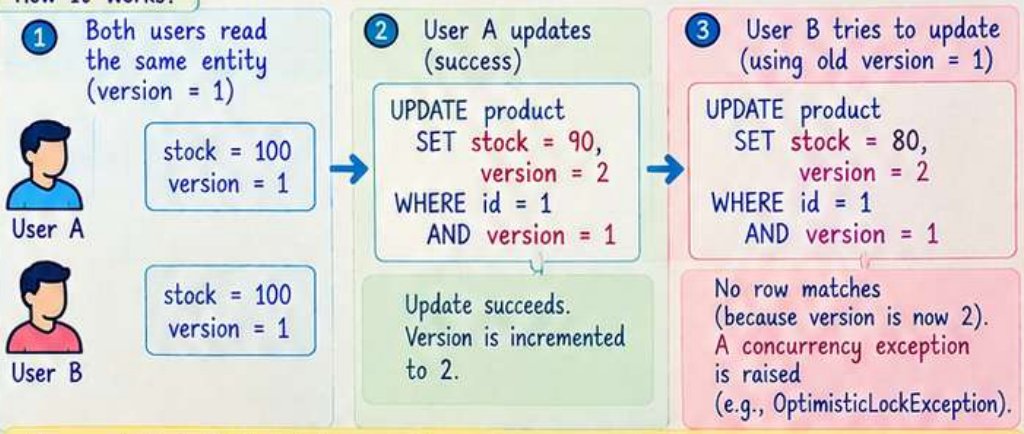
```
@Entity
class Product {
    @Id
    private Long id;

    private int stock;

    @Version
    private int version;
}
```

The @Version field is managed by JPA. You should not set its value manually.

How It Works?



No row lock is held just because the entity was read. The version check detects concurrent modification during update.

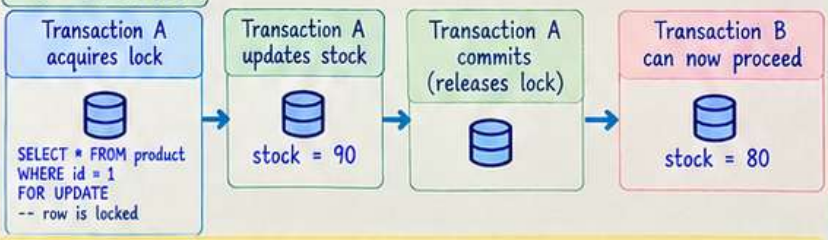
3 Pessimistic Locking (Database Row Lock)

Repository Method

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM Product p WHERE p.id = :id")
Optional<Product> findByIdForUpdate(
        @Param("id") Long id);
```

Commonly translates to a database row-locking operation such as SELECT ... FOR UPDATE, depending on the database and provider.

How It Works?



While Transaction A holds the lock, other transactions trying to acquire a conflicting lock will wait or may time out, depending on the database and configuration.

4 Comparison

Aspect	Optimistic Locking	Pessimistic Locking
Mechanism	@Version	@Lock with appropriate lock mode
When conflict is detected	During write (version check)	During read (locks the row)
Blocking behavior	Other transactions need not wait on a row lock.	Contending work can wait or time out.
Best suited for	Low-contention updates (e.g., user profiles, general data).	High-contention, critical operations (e.g., inventory).
Main advantage	Better concurrency, no unnecessary blocking.	Stronger isolation for critical sections.
Main drawback	Update may fail and need retry (OptimisticLockException).	Can reduce concurrency due to locking and possible timeouts/deadlocks.

5 Practical Decision Guide

- User profile or ordinary updates → Consider optimistic locking.
- Critical inventory adjustment with frequent contention → Consider pessimistic locking.
- Deadlock prevention → Acquire multiple locks in a consistent order and keep transactions short.

⚠ Both approaches have trade-offs. Choose based on your use case, expected contention, and consistency requirements.

HIBERNATE CACHE — REDUCE REPEATED DATABASE WORK

- Hibernate provides a first-level cache (L1) and an optional second-level cache (L2) to reduce database access. There is also a separate query cache for caching query result information.

1 First-Level Cache (L1)

Always present in the persistence context.

```
@Transactional
public void example() {
    User u1 = userRepository.findById(1L)
            .orElseThrow();
    User u2 = userRepository.findById(1L)
            .orElseThrow();
    System.out.println(u1 == u2); // true
}
```

- Entity lookups by identity can reuse the managed instance in the same persistence context.
- Repeated findById for the same managed entity normally avoids another database read.
- Scope is the persistence context // session, commonly transaction-scoped in typical service code.
- No separate second-level cache configuration is needed.

How L1 Works (Within One Transaction)

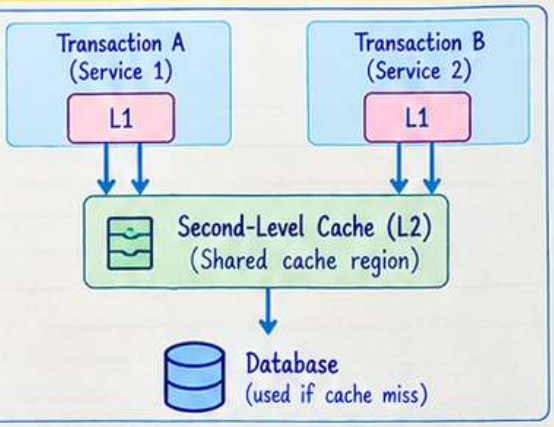
Persistence Context (L1 Cache)

Entity Id	Managed Instance
1	[User object]

1. findById(1) → DB
   SELECT * FROM user WHERE id = 1 (only first time)
   Stored in L1 cache
2. findById(1)
   ↳ Returns same instance from L1 (no DB query)

2 Second-Level Cache (L2)

Shared across persistence contexts; requires configuration.

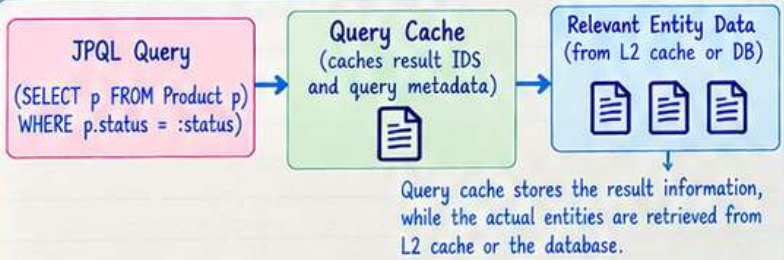



```
@Entity
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
class Product {
    @Id
    private Long id;
}   // other fields
```

Provider-specific Hibernate caching annotation; configure a supported cache region factory and cache provider before relying on it.

3 Query Cache

Entity caching by ID and caching query result information are different features. Query caching must be enabled and configured separately and requires correct invalidation behavior.



Query cache stores the result information, while the actual entities are retrieved from L2 cache or the database.

```
spring.jpa.properties.hibernate.cache.use_query_cache = true
spring.jpa.properties.hibernate.cache.use_second_level_cache = true
# plus cache region factory and provider configuration
```

4 Cache Concurrency Strategies

Strategy	When to use / Notes
READ_ONLY	• For data that never changes. • Best performance; no update support.
NONSTRICT_READ_WRITE	• When brief stale data can be acceptable. • Lower overhead; suited to less critical data.
READ_WRITE	• Stronger coordination for changing data. • Uses soft locks to reduce stale reads.
TRANSACTIONAL	• Transactional cache coordination. • Requires a JTA transaction manager and a cache provider that supports it.

5 Caching Pitfalls

1. Cache data selectively, based on measured access patterns.
2. Frequently changing data may not benefit from caching.
3. Direct SQL or bulk modifications can leave cached state stale unless invalidation is handled correctly.
4. Cache hits, misses, invalidation, and memory use must be monitored.
5. Do not confuse Spring's @Cacheable (method-level caching) with Hibernate's entity-cache annotation (@Cache).

6 Key Takeaways

- ★ First-level cache belongs to the persistence context; second-level cache is shared across persistence contexts.
- ★ L1 is always enabled; L2 and query cache require configuration.
- ★ Use the appropriate concurrency strategy based on your data and consistency requirements.
- ★ Monitor cache behavior in real environments.
- ★ Caching reduces database work, but it does not remove the need for correct transactional and data consistency design.

SPRING APPLICATION CACHING

Spring Data JPA Notes

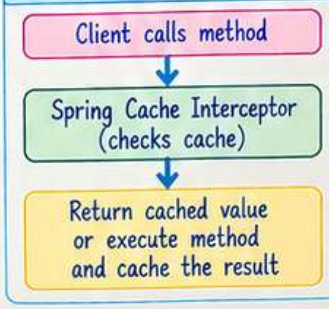
- Spring's application-level caching abstraction allows method results to be cached, reducing repeated database calls and improving performance. It works with different cache providers (e.g., Caffeine, Redis, Ehcache).

1 Enable It

```
@EnableCaching
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

- Spring's caching infrastructure must be enabled using @EnableCaching.
- Configure a suitable cache provider for the intended application (e.g., Caffeine, Redis, Ehcache).
- By default, Spring uses a simple in-memory cache (ConcurrentMapCache) if no provider is configured.

How It Works (High Level)

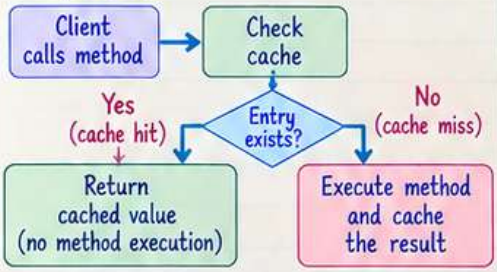


2 Three Main Annotations

@Cacheable — Read / Lookup

`@Cacheable("users")`

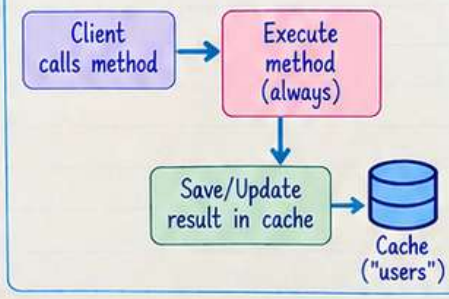
- If an entry exists for the key, return it from the cache.
- Otherwise execute the method and cache the result.
- Use for suitable read/lookup operations.



@CachePut — Update / Refresh

`@CachePut(value = "users", key = "#result.id")`

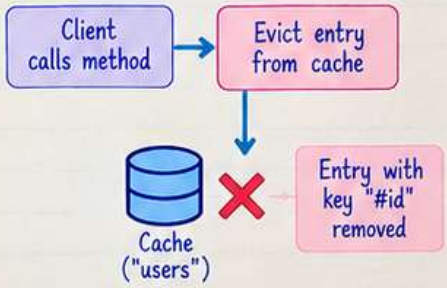
- Always execute the method and place the returned result in the cache under the selected key.
- Use when the chosen key correctly identifies the updated value.



@CacheEvict — Remove / Invalidate

`@CacheEvict(value = "users", key = "#id")`

- Remove a matching cache entry so stale data is not returned.
- Use when application changes make the cached data outdated.



3 Default Cache Keys

Single Argument

```
@Cacheable("users")
public User getUserById(Long id) { ... }
```

Key = argument value (e.g., 1L)

Multiple Arguments

```
@Cacheable("users")
public User getUser(String name,
                    String email) { ... }
```

Key = SimpleKey(name, email) (compound key)

No Arguments

```
@Cacheable("config")
public List<Config> getAllConfigs() { ... }
```

Key = SimpleKey.EMPTY (uses a shared key)

4 Custom SpEL Keys

```
@Cacheable(value = "users", key = "#id")
public User getUserById(Long id) {
    // method logic
}
```

Other useful SpEL examples:

- #email → method parameter
- #user.id → property of an argument
- #root.args[0] → first method argument
- #result.id → property of the returned result
- T(java.util.UUID).randomUUID() → expression

Prefer stable, meaningful keys such as IDs or unique business identifiers.

5 Common Mistakes

1. @Cacheable can skip method execution on a hit; never use it as a substitute for executing a required database write.
2. Updating the database does not automatically make every unrelated cache entry fresh.
3. Evict or update entries when application changes make them stale.
4. Default key generation depends on the method arguments.

6 Key Takeaways

- Enable caching with @EnableCaching.
- Use @Cacheable for read/lookup scenarios.
- Use @CachePut when you want to always update the cache with the returned value.
- Use @CacheEvict to remove stale entries.
- Choose stable and meaningful cache keys.
- Configure an appropriate cache provider based on your application needs.

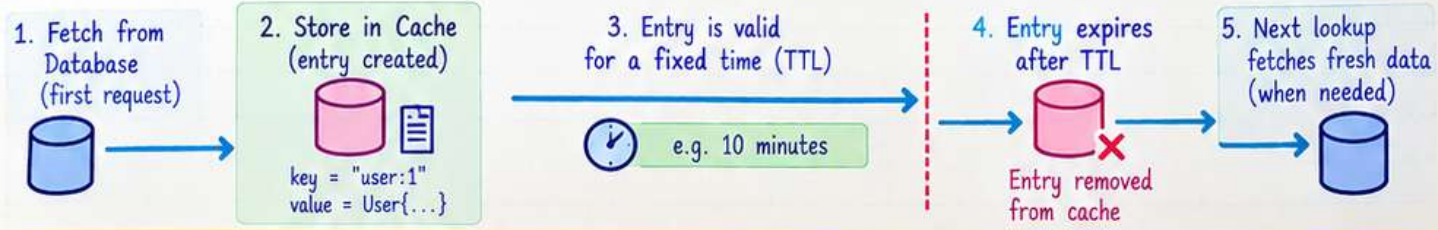
CAFFEINE — CONTROL CACHE LIFETIME

Spring Data JPA Notes

- Caffeine is a high-performance, in-memory cache provider commonly used with Spring's caching abstraction. It provides powerful features like expiration policies and size-based eviction.

1 What Is TTL (Time-To-Live)?

TTL defines how long a cached entry remains valid before it expires.



- A simple cache provider may retain entries until eviction or application restart.
- Production caching needs deliberate lifetime (TTL) and memory policies to avoid stale data and uncontrolled memory growth.

2 Caffeine Configuration

```
Caffeine.newBuilder()
      .expireAfterWrite(10, TimeUnit.MINUTES)
      .maximumSize(5000)
      .build();
```

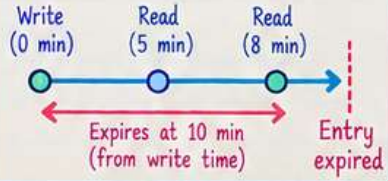
`expireAfterWrite(10, MINUTES)` → Expiration measured from the last write/update. Reads do not reset the timer.

`maximumSize(5000)` → Bounds the number of retained entries. It does not guarantee an exact memory-byte limit (since entry size varies).

3 Expiration Policy Comparison

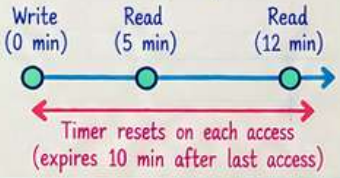
expireAfterWrite(10, MINUTES)

- Timer begins at write/update.
- Reads do NOT reset the expiry timer.
- The entry expires 10 minutes after it was written, regardless of how often it is accessed.



expireAfterAccess(10, MINUTES)

- Timer begins at write or read (access).
- Each access (read or write) resets the expiry timer.
- Active entries can stay in the cache longer if they are accessed frequently.



Choose the appropriate policy based on freshness requirements and traffic patterns.

4 Example Cache Policies

Cache Name	TTL / Expiration	Why / Use Case
companies	10 minutes (expireAfterWrite)	Company data changes moderately; refresh periodically.
jobs	10 minutes (expireAfterWrite)	Job listings often updated; keep data fresh.
roles	1 day (expireAfterWrite)	Relatively static reference data; less frequent changes.

- These are illustrative policy choices, not universal defaults. Choose TTL based on your actual business requirements and data update frequency.

5 Best Practices

1. Set a suitable maximum size to avoid unbounded growth.
2. Select expiration policies (expireAfterWrite or expireAfterAccess) deliberately.
3. Evict or update data after relevant writes.
4. Cache only after measuring a real performance bottleneck.
5. Monitor cache hit rate, misses, invalidation, and memory use.
6. In multi-instance deployments, use an appropriate shared-cache or consistency strategy when cross-instance cache coherence matters.

6 Final Comparison

Spring Application Cache (Caffeine, etc.)

- Used with @Cacheable, @CachePut, @CacheEvict.
- Caches method results in the application layer.
- Works with various providers (Caffeine, Redis, Ehcache).
- Configured through Spring's caching abstraction.
- Suitable for caching read/lookup results in service methods.

Hibernate L2 Cache

- Entity and query persistence caching.
- Managed by the JPA provider (e.g., Hibernate).
- Configured using provider-specific settings and annotations (e.g., @Cache).
- Works at the entity/query level within JPA.
- Focuses on persistence context and entity data semantics.

SPRING DATA JPA — INTERVIEW QUICK REVISION

Spring Data JPA Notes

- A final revision sheet covering the most important concepts, differences, annotations, problems and best practices for Spring Data JPA interviews.

A CORE CONCEPTS & QUESTIONS

What to remember?

1 Technology Roles

JPA	• Specification (Jakarta Persistence API) • Defines ORM standards and APIs.
Hibernate	• Persistence implementation • Implements JPA specification.
Spring Data JPA	• Repository abstraction • Simplifies data access with repository interfaces and query methods.
JDBC	• Lower-level database communication • Manual SQL and result mapping.

2 Entity Essentials

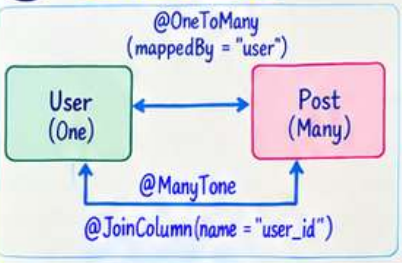
```
@Entity
public class User {
    @Id
    @GeneratedValue
    private Long id;

    @Column(nullable = false)
    private String name;

    @Enumerated(EnumType.STRING)
    private UserStatus status;
}
```

- Use @Entity to mark the class as a JPA entity.
- Every entity must have a primary key (@Id).
- Use @GeneratedValue for auto generation (IDENTITY, SEQUENCE, etc.).
- Use @Column to configure column details (nullable, length, unique, etc.).
- Prefer @Enumerated(EnumType.STRING) for enums.
- Provide a no-args constructor (required by JPA).

3 Relationships



- @JoinColumn is used on the owning side to define the foreign key.
- mappedBy is on the inverse side (no separate FK column).
- Keep both sides in sync in bidirectional relationships (add/remove in helper methods).

4 Querying Options

Derived Methods	Simple queries based on method names. e.g., findByNameAndStatus(..)
JPQL	Entity and field-oriented queries (database-independent).
Native SQL	Database-specific requirements (use when needed).
Projections	Fetch only selected fields (interface/DTO/dynamic).
Page vs Slice	Page → total count, pagination metadata. Slice → next-page information (no total count).

B PRODUCTION PITFALLS & KEY TOPICS

Common in real projects

5 Fetching (Eager vs Lazy)

- Default EAGER: @ManyToOne and @OneToOne
- Default LAZY: @OneToMany and @ManyToMany
- Prefer explicit fetch plans (JOIN FETCH, @EntityGraph, etc.) based on use case and performance requirements.

Fetch strategy is not the same as query strategy.

6 N+1 Problem

- Occurs when related entities are loaded with repeated queries (1 + N).
- Detect using logs (spring.jpa.show-sql=true) or query tools.
- Solutions: JOIN FETCH, @EntityGraph, @BatchSize, SUBSELECT.

Check query count and generated SQL.

7 Persistence Context / Entity States

Transient (new) → Managed (in persistence context) → Detached (after session ends) → Removed (scheduled for deletion)

- Managed entity: changes can be tracked by dirty checking.
- Detached entity: ordinary setter changes are not automatically persisted.
- findById() returns Optional<T>.

8 Transactions and Concurrency

- @Transactional uses proxy-based behavior in common configurations.
- REQUIRED is the default propagation.
- Rollback occurs on unchecked exceptions and Errors by default.
- @Version → optimistic locking (detects concurrent updates).
- @Lock → database lock mode for pessimistic locking.

- A database transaction does not automatically reverse external side effects (e.g., payment APIs).

9 Caching (Hibernate + Spring) and Caffeine

First-level cache	Persistence context (session/transaction scoped).
Second-level cache	Shared Hibernate cache (requires configuration).
@Cacheable	Read-through method caching (returns cached value if present).
@CachePut	Always execute method and updates cache.
@CacheEvict	Removes cache entry to avoid stale data.
TTL (Caffeine)	Use expireAfterWrite / expireAfterAccess and maximumSize to control lifetime and memory usage.

C ADVANCED INTERVIEW EXTRAS

High-value topics

1 EntityManager and Persistence Context

- EntityManager operations: find(), persist(), merge(), remove, createQuery().
- Persistence context tracks managed entities and participates in dirty checking.
- Usually transaction-scoped in Spring applications.

2 Specification Pattern (Dynamic Queries)

- Use Specification<T> for optional filters (e.g., name, status, date range).
- Repository extends JpaSpecificationExecutor<T>.

Optional filters → Specifications (build predicates) → and()/or() (combine) → Dynamic query

3 @MappedSuperclass and Soft Deletion

- @MappedSuperclass: mapped fields are inherited by entity subclasses; base class is not an entity table.
- Soft delete: keep a deleted flag or timestamp instead of physically removing the record.
- Hibernate-specific options: @SQLDelete and visibility filters (e.g., @Where). Verify behavior for the Hibernate version in use.

FINAL INTERVIEW MEMORY STRIP

1. Fetch strategy is not the same as query strategy.
2. Database transactions do not automatically reverse independent external side effects.
3. Optimize based on generated SQL, query counts, and measured performance.

20 / 20 END OF COLLECTION